UNIVERSIDAD
Panamericana

PROGRAMACIÓN AVANZADA

Sesión 12

Torres de Hanoi en Arte ASCII

→ NOTAS DE CLASE

Universidad Panamericana
Dr. Miguel Alcaraz Rivera • malcaraz@up.edu.mx

2026

Índice

La leyenda y el problema

En 1883, el matemático francés **Édouard Lucas** lanzó al mercado un juguete llamado *La Tour d'Hanoï* — un rompecabezas de ocho discos de madera de distinto tamaño apilados en una de tres estacas. Para justificar el precio, lo acompañó con una leyenda apócrifa:

> *En un templo de Benarés, en la India, los sacerdotes del dios Brahma mueven en silencio 64 discos de oro puro entre tres agujas de diamante, una y otra vez, sin descanso. Cuando terminen de moverlos todos a la tercera aguja, el mundo llegará a su fin.*

Lucas la inventó él mismo para darle misterio al juguete, pero la pregunta matemática que esconde es completamente real: **¿cuántos movimientos mínimos se necesitan para mover n discos?**

Para $n = 64$ discos, el mínimo de movimientos es $2^{64} - 1 \approx 1.8 \times 10^{19}$. Si los sacerdotes movieran un disco por segundo, sin parar, tardarían aproximadamente **585 mil millones de años** — unas 42 veces la edad actual del universo. El mundo, al parecer, tiene tiempo.
Referencia: Lucas, É. *Récréations Mathématiques*. Gauthier-Villars, 1883.

Las reglas del juego

El problema tiene exactamente **tres reglas**:

1. Solo puedes mover **un disco a la vez**.
2. Solo puedes mover el disco que está en el **tope** de una torre.
3. **Nunca** puedes colocar un disco sobre uno más pequeño.

El objetivo es trasladar todos los discos de la torre **A** a la torre **C**, usando la torre **B** como auxiliar, respetando las tres reglas en todo momento.

```
Estado inicial:        Estado final:

    |       |       |         |       |       |
   ===      |       |         |       |      ===
  =====     |       |         |       |     =====
 =======    |       |         |       |    =======
=========   |       |         |       |   =========
    A       B       C             A       B       C
```

¿Cuántos movimientos se necesitan?

Antes de programar nada, vale la pena entender cuánto trabajo implica el problema. Para n discos, el **mínimo de movimientos** es:

$$M(n) = 2^n - 1$$

- ## ¿De dónde viene esa fórmula?

  Piensa recursivamente:

  - Para mover el disco más grande (el n) de A a C, necesitas que los $n-1$ discos que están encima **ya estén en B**.
  - Mover esos $n-1$ discos de A a B cuesta $M(n-1)$ movimientos.
  - Mover el disco n de A a C cuesta **1** movimiento.
  - Mover los $n-1$ discos de B a C cuesta $M(n-1)$ movimientos.

  La recurrencia es:

$$M(1) = 1 \qquad M(n) = 2\,M(n-1) + 1$$

  Resolviendo:

$$M(n) = 2^n - 1$$

Discos	$2^n - 1$	**Movimientos**
1	$2^1 - 1$	1
2	$2^2 - 1$	3
3	$2^3 - 1$	7
4	$2^4 - 1$	15
8	$2^8 - 1$	255
64	$2^{64} - 1$	$\approx 1.8 \times 10^{19}$

✔ Recuerda

La recurrencia $M(n) = 2\,M(n-1) + 1$ **es** la descripción del algoritmo recursivo que implementarás en la sesión 13. Hoy no lo programas: tu equipo mueve los discos a mano (el usuario escribe los movimientos). En la sesión 13 harás que la computadora los calcule sola.

Hay una propiedad fascinante: en la secuencia óptima de movimientos, el disco que se mueve en el paso k es el mismo que el bit menos significativo diferente de cero en la representación binaria de k.

Para $n = 3$, los primeros movimientos son:

Paso k	k **en binario**	**Disco**
1	001	disco 1 (bit 0)
2	010	disco 2 (bit 1)
3	011	disco 1 (bit 0)
4	100	disco 3 (bit 2)
5	101	disco 1 (bit 0)
6	110	disco 2 (bit 1)
7	111	disco 1 (bit 0)

> Esta propiedad conecta el problema con el *código Gray* (código binario reflejado), usado en electrónica para evitar glitches al incrementar contadores. El espacio de estados de Hanoi también tiene la forma del **triángulo de Sierpiński** — uno de los fractales más famosos de las matemáticas.
> *Referencia:* Gardner, M. *Mathematical Games*. Scientific American, 1958.

Arte ASCII como interfaz

El programa que van a construir hoy no usa ventanas ni botones: toda su interfaz vive en la terminal, hecha de caracteres. Eso es **arte ASCII aplicado** a la programación.

• ¿Por qué importa el diseño visual del programa?

Un programa que funciona pero es difícil de leer es un programa incompleto. La claridad visual de la salida es parte de la calidad del código. Cuando la interfaz de una herramienta de terminal es buena, el usuario sabe inmediatamente qué está pasando sin leer manual.

Aquí está la diferencia entre un estado impreso de forma mínima y uno bien diseñado:

```
Sin diseño:                Con diseño:
A: [4,3]                       |           |           |
B: [2]                        ===          |           |
C: [1]                       =====         |           |
                             =======      ===          |
                             =========   =====        ===
                             =========  =========  =========
                                  A          B          C
```

Ambos contienen la misma información, pero el segundo la hace inmediatamente comprensible.

• Algunos caracteres útiles para diseño en terminal

Categoría	Caracteres	Uso típico
Líneas simples	`| - =`	Postes y bases básicas
Bloques de sombra	`U+2591..2588`	Discos rellenos con gradiente
Líneas dobles	`U+2550..256C`	Marcos y bordes elegantes
Flechas	`U+2192..21D0` (->, <-)	Indicadores de movimiento
Figuras geométricas	`U+25B2,25BC..` (^ v *)	Decoración de discos
Relleno alternativo	`# * ~ ^`	Diseños personalizados

> Los caracteres Unicode de bloques (`U+2591..2588`, `U+2550..256C`, etc.) funcionan en la mayoría de terminales modernas (macOS, Linux y Windows Terminal). Si tu terminal no los muestra bien, usa los caracteres ASCII básicos (= | #).

> Antes de diseñar, comprueba que tu terminal soporte lo que quieres usar.

La actividad de hoy

Trabajan en **equipos de tres personas**. El objetivo es construir un programa que muestre las Torres de Hanoi en arte ASCII y permita al usuario jugar.

- **Lo que deben implementar**

1. **Función de dibujo.** Dado el estado de las tres torres (qué discos tiene cada una), imprimir la representación visual en la consola.
2. **Función de movimiento.** Recibir una torre de origen y una de destino, validar que el movimiento sea legal y ejecutarlo.
3. **Interfaz de consola.** Un ciclo donde el usuario escribe el origen y destino de cada movimiento, el programa dibuja el estado actualizado y detecta cuando el jugador ganó.

- **Lo que deben entregar en Canvas**

- El **código fuente** (archivo `.cpp`) con los tres integrantes del equipo en un comentario al inicio.
- **Capturas de pantalla o fotos** mostrando las torres en al menos **4 estados distintos**:
  1. Estado inicial (todos los discos en la torre A).
  2. Un estado intermedio con los discos distribuidos.
  3. Un estado casi final.
  4. Estado final (todos los discos en la torre C).

> **No implementen el algoritmo que resuelve Hanoi automáticamente.** Eso es el tema de la sesión 13. Hoy, los movimientos los elige el usuario escribiendo origen y destino. Si el programa se resuelve solo, no cumple con el objetivo de la actividad.

- ## Criterios de evaluación

Criterio	Descripción	Peso
Funcionalidad	El programa dibuja las torres correctamente y las actualiza tras cada movimiento	40 %
Validación	Rechaza movimientos ilegales con un mensaje claro	20 %
Condición de victoria	Detecta cuando todos los discos están en C	10 %
Diseño visual	Las torres son claras, el arte es consistente y estéticamente agradable	20 %
Legibilidad del código	Nombres de funciones y variables descriptivos, estructura clara	10 %
	Total	100 %

El **código ganador del concurso de la sesión 13** se usará como base para implementar el algoritmo recursivo. Eso significa que vale la pena que el código esté bien estructurado desde hoy — no solo bonito visualmente. Un diseño elegante con código ininteligible no será fácil de extender en la siguiente sesión.

Análisis previo: ejercicios a mano

Antes de ponerte a programar, resuelve estos ejercicios con lápiz y papel. Te ayudarán a entender el problema desde adentro.

1. Dibuja en papel el estado de las torres después de cada uno de los 7 movimientos óptimos para mover 3 discos de A a C. Numera cada estado del 0 (inicial) al 7 (final).

2. Verifica con la fórmula que $M(3) = 7$. Luego aplica la recurrencia para calcular $M(4)$ y $M(5)$.

3. Indica cuántos frames se apilarían en el call stack cuando el algoritmo recursivo calcule la solución para $n = 4$. *(Pista: la profundidad del árbol de recursión es n.)*

4. ¿Cuál es el disco que siempre se mueve en el paso k si k es potencia de 2? Justifica con la tabla de la sección 3.

5. **Desafío:** Con 4 agujas en lugar de 3, ¿necesitarías más o menos movimientos para mover n discos? Razona sin calcular.

Resumen

✔ Recuerda

- Las Torres de Hanoi fueron inventadas por Édouard Lucas en 1883 como juguete comercial; la leyenda del templo es apócrifa.
- Tres reglas: un disco a la vez, solo el tope, nunca un disco grande sobre uno pequeño.
- Mínimo de movimientos: $M(n) = 2^n - 1$. Recurrencia: $M(n) = 2\,M(n-1) + 1$.
- La secuencia óptima está relacionada con el código binario y el triángulo de Sierpiński.
- Hoy implementas la interfaz visual y los movimientos del usuario. La sesión 13 agrega el algoritmo recursivo que lo resuelve solo.